Instant Side-Channel Vulnerability Estimation of Digital Designs

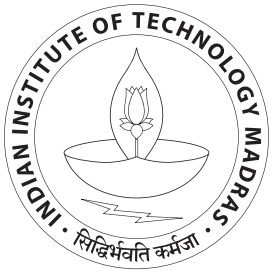
A Project Report

submitted by

AYALUR VEDPURISWAR LAKSHMY

in partial fulfilment of the requirements
for the award of the degree of

MASTER OF TECHNOLOGY



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY, MADRAS.

June 2021

THESIS CERTIFICATE

This is to certify that the thesis entitled **Instant Side-Channel Vulnerability Estimation of Digital Designs**, submitted by **Ayalur Vedpuriswar Lakshmy**, to the Indian Institute of Technology, Madras, for the award of the degree of **Master of Technology**, is a bona fide record of the research work carried out by her under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. Chester Rebeiro
Research Guide
Assistant Professor
Dept. of Computer Science and Engineering
IIT-Madras, 600 036

Place: Chennai

Date: June 21, 2021

ACKNOWLEDGEMENTS

I would like to thank my guide, Prof. Chester, for giving me the opportunity to work on this project, and for his guidance and support throughout the project. I would like to thank Milind and Arsath for helping me during the initial stages of the project. I am also grateful to the Qualcomm Innovation Fellowship mentors, Venkatesh Sir, Meet Sir and Balaji Sir, for their valuable inputs and suggestions during the course of the project. I would also like to acknowledge the Samsung - IITM Pravartak Fellowship for their support. Last but not the least, I would like to thank my parents for their constant moral support and motivation.

ABSTRACT

KEYWORDS: Power Side-Channel, Leakage Score, RTL, Gate-level Netlist, Verilog, Signal Probability, Conditional Signal Probability

Power side-channel attacks involve analysing the power consumption patterns of an electronic device in order to glean information about sensitive data being processed by the device, such as secret keys and passwords, or operations being performed by the device, such as web browsing activity. Contemporary techniques to estimate the power side-channel vulnerability of an electronic device are usually performed on manufactured devices, which is too late in the design cycle to take any corrective measures. Even if performed earlier in the design stage, these techniques are extremely time-consuming. We present a framework to estimate the power side-channel vulnerability of digital circuit designs at the RTL or gate-level netlist stage, using mathematical models based on signal probability. These models can be easily computed resulting in instant results. For example, while contemporary works to evaluate a digital design can take several hours, our framework can do so within a few minutes. Moreover, the side-channel leakage estimation is provided at a fine granularity and with an accuracy comparable with the state-of-the-art evaluation tools.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS

SCA	Side-Channel Attack
PSCA	Power Side-Channel Attack
PLAN	Power attack Leakage ANalyzer
RTL	Register Transfer Level
SVF	Side-channel Vulnerability Factor
SoC	System-on-Chip
AES	Advanced Encryption Standard

NOTATION

$SP(sig)$	Signal probability of a signal *sig*
$SP_{A=0}(sig)$	Conditional signal probability of a signal *sig* with respect to a reference signal A taking the value 0
$SP_{A=1}(sig)$	Conditional signal probability of a signal *sig* with respect to a reference signal A taking the value 1
$L(sig)$	Power side-channel leakage score of a signal *sig*

CHAPTER 1

INTRODUCTION

In the past few decades, Side-Channel Attacks (SCA) on electronic devices have become increasingly common. These attacks exploit the accidental leakage of information through the inherent physical interactions between an electronic device and its surroundings, for example, through timing channels, power dissipation, electromagnetic radiations, sound emanations, and so on.

Power side-channel attacks (PSCA) [Kocher *et al.*, 1999] can be used to indirectly reveal the data being processed or the operations being performed by an electronic device, by analysing its dynamic power consumption patterns. A typical power side-channel attack, such as Differential Power Analysis (DPA), targets a cryptographic device, for instance, a smart card reader. An oscilloscope is used to measure the power traces of the device for a range of known input values. These power traces are then analysed using power models, such as Hamming Weight or Hamming Distance models [Mangard *et al.*, 2008], and statistical metrics are applied, in order to steal secret keys and break cryptographic ciphers. It has been shown practically that simple power side-channel attacks can compromise strong ciphers like RSA and AES in a matter of a few minutes [Mangard, 2002]. Over the years, many more variants of power side-channel attacks have been developed, including Correlation Power Analysis (CPA) [Brier *et al.*, 2004], Inferential Power Analysis (IPA) [Fahn and Pearson, 1999], Partitioning Power Analysis (PPA) [Le *et al.*, 2006] and Mutual Information Analysis (MIA) [Gierlichs *et al.*, 2008].

The use of power side-channel attacks goes even beyond cryptography. For

instance, power side-channel attacks can be applied for finding passwords [Oren and Shamir, 2007], reverse engineering software [Moradi *et al.*, 2012, 2013], reverse engineering Machine Learning and Deep Learning algorithms [Batina *et al.*, 2019; Dubey *et al.*, 2020], logging keystrokes, fingerprinting website activities [Qin and Yue, 2018], and so on. Thus, power side-channel attacks can result in leakage of sensitive information if electronic devices are not properly protected against them.

In order to do that, we must be able to estimate the power side-channel vulnerability of an electronic device. Most of the prior works in this field focus on testing an already manufactured device or chip for vulnerabilities [Veshchikov, 2014; Veshchikov and Guilley, 2017; McCann *et al.*, 2017], typically by attempting an attack on the device, i.e., by collecting the power traces and analysing them using statistical metrics to glean sensitive information from the device. If the device is found not to satisfy the required security standards, then the design would have to be revisited, which turns out to be very expensive due to the need for re-fabrication. Thus, this approach is too late in the design cycle to take any corrective measures. Some of the more recent works perform power side-channel vulnerability estimation earlier in the development process, i.e., in the design stage itself, by modelling the device and performing simulations to obtain the power traces, which are then analysed using power models in order to estimate the power side-channel vulnerability [Slpsk *et al.*, 2019; He *et al.*, 2019; KF *et al.*, 2020; Zoni *et al.*, 2018; Nahiyan *et al.*, 2020; Šijačić *et al.*, 2020]. This approach enables hardware designers to assess the security of the design and make appropriate modifications at the design stage itself, so that, after fabrication, the device would be sufficiently protected against attacks. However, this approach is relatively less accurate than the post-fabrication vulnerability assessment, and is also extremely time-consuming due to the large number of simulations which are required to be

performed. Moreover, most of the power side-channel vulnerability assessment tools developed so far are mostly applicable to cryptographic designs; very few tools are capable of analysing non-cryptographic (general purpose) designs, which might also be targets of side-channel attacks.

Our work addresses the following question: *Can we efficiently estimate the power side-channel vulnerability of a general hardware design without involving lengthy simulations?* We have developed a tool that performs power side-channel vulnerability estimation of any hardware design at the pre-fabrication (RTL or gate-level netlist) stage, sufficiently early in the design cycle, so that targeted and appropriate countermeasures can be applied to the vulnerable portions before manufacturing. Our tool quantifies the leakage of information from different parts of the design using mathematical models based on signal probabilities. Our key contributions are as follows:

- We have identified and mathematically formulated the relationship between power side-channel leakage and signal probability (Chapter 5).
- We have developed a framework for efficiently assessing the power side-channel leakage of general digital circuit designs at the RTL or gate-level netlist stage (Chapter 6).
- We have applied our framework to compute the power side-channel leakage scores of both cryptographic and non-cryptographic digital circuit designs, with an accuracy comparable with state-of-the-art evaluation tools. We have also used our framework to evaluate a modern open-source System-on-Chip (SoC) at a fine granularity, in a matter of a few hours (as compared to conventional tools which could take several days) (Chapter 7).

CHAPTER 2

BACKGROUND

2.1 Terminology

- The **reference signal** is a signal in a digital circuit design, which is likely to be targeted during a power side-channel attack to reveal sensitive information, such as cryptographic keys, passwords, etc. For example, in a cryptographic design, the reference signal could be the result of the key-xor operation between the secret key and the plain text.
- The **leakage score** of a signal in the digital circuit design is a value in the range $[0, 1]$ which quantifies the amount of information the signal carries with respect to the reference signal. A leakage score of 0 implies that the signal reveals no information about the reference signal, while a leakage score of 1 implies that the signal completely "leaks" information about the reference signal.

2.2 Signal Probability

The **signal probability** (SP) of a signal sig in a circuit C is defined as the probability with which sig takes the value 1, taken over all possible values of the primary inputs of C. Mathematically, this can be expressed as:

$$SP(sig) = \Pr(sig = 1)$$

Signal probabilities can be calculated using the truth table of the signals in a circuit, since a truth table captures the entire range of values that can be taken by the signals in the circuit. For instance, let us consider a simple 2-input OR gate,

whose truth table is shown in Table 2.1. Both the input signals A and B take the value 1 in two out of four possible cases, so the signal probability of both A and B is 0.5 each. The output signal *Out* takes the value 1 three times out of the four possible cases, so the signal probability of *Out* is 0.75.

Table 2.1: Truth Table of a 2-input *OR* gate

A	B	*Out*
0	0	0
0	1	1
1	0	1
1	1	1

While the truth table-based approach can be used to calculate signal probabilities accurately, it does not scale well to larger circuits, since the size of the truth table of a circuit is exponential in the number of inputs to the circuit.

An alternative approach is to incrementally calculate the signal probabilities of signals based on their logical expressions. The ***incremental signal probability formulae*** (Table 2.2) are derived by applying basic probability rules based on the function of the logic gates, with the assumption that the inputs to the gates are independent of each other (refer to Appendix A for more details).

Table 2.2: Incremental Signal Probability Expressions

Logical Expression	Signal Probability
$\texttt{NOT}(A)$	$1 - SP(A)$
$\texttt{AND}(A, B)$	$SP(A) \cdot SP(B)$
$\texttt{OR}(A, B)$	$SP(A) + SP(B) - SP(A) \cdot SP(B)$
$\texttt{XOR}(A, B)$	$SP(A) + SP(B) - 2 \cdot SP(A) \cdot SP(B)$

In order to calculate the signal probabilities of the signals in a given digital circuit design C, we can start by assigning the initial signal probabilities of the

inputs to the design, and then applying the incremental signal probability formulae (based on the logic gate involved) to each remaining signal in the design.

We note that the incremental signal probability calculation assumes that the inputs to each logic gate in the circuit are independent. However, this assumption may not always be valid, especially in case of ***reconvergent fanouts***. A reconvergent fanout refers to a signal which branches out into two or more signals proceeding along different paths, which reconverge at some logic gate later in the circuit. Hence, the incremental signal probability algorithm results in accurate signal probability values only if the circuit design has no reconvergent fanouts, otherwise, the signal probability values are not accurate but close estimates. However, this assumption allows signal probability calculation of all the signals in the design in linear time $O(m + n)$, where m is the number of logic gates and n the number of signals in C.

2.3 Conditional Signal Probability

The **conditional signal probability** of a signal *sig* with respect to a reference signal A is the probability of *sig* taking the value 1 given that A takes the value 0 (or 1). Mathematically, this can be expressed as:

$$SP_{A=0}(sig) = \Pr(sig = 1 \,|\, A = 0)$$

$$SP_{A=1}(sig) = \Pr(sig = 1 \,|\, A = 1)$$

CHAPTER 3

RELATED WORK

Power side-channel vulnerability estimation can be performed using either ***post-silicon*** or ***pre-silicon*** techniques [Buhan *et al.*, 2021].

3.1 Post-silicon Techniques

Post-silicon techniques evaluate an already manufactured electronic device for resistance against power side-channel attacks. Conventionally, this involves collecting the power traces via physical measurements while the device executes cryptographic (or other) programs. Then, the collected power traces are analysed using statistical metrics (such as Side-channel Vulnerability Factor (SVF) [Demme *et al.*, 2012], Normalised Inter-Class Variance (NICV) [Bhasin *et al.*, 2014], Test Vector Leakage Assessment (TVLA) [Cooper *et al.*, 2013], and so on). Since collecting power traces physically is quite time-consuming, modern techniques do this by running software implementations of algorithms on a simulator of the device. For example, SILK [Veshchikov, 2014] evaluates the high-level source code of an AES cryptographic algorithm by simulating its execution on an AVR microcontroller. SAVRASCA [Veshchikov and Guilley, 2017] is another tool which works with compiled binary files of cryptographic algorithms for AVR microcontrollers. The more recent ELMO [McCann *et al.*, 2017] proposes a simulator for ARM Cortex M0 processors and evaluates an AES implementation.

The main advantage of post-silicon techniques is that they can provide accurate side-channel leakage estimation, without requiring much knowledge of the internal details of the design specifications of the device. However, all these techniques involve statistical metrics for their analysis, which requires the collection of a large number of power traces, either by physical measurements or via simulations, both of which are extremely time-consuming. For instance, it takes 275 hours to perform sufficient number of simulations to analyse the power side-channel resistance of a stripped-down DES (Data Encryption Standard) cryptographic algorithm [Tiri and Verbauwhede, 2003]. Even after collecting the power traces and evaluating the side-channel leakage, it is difficult to identify the root cause of the leakage since the design information is not available in the post-silicon stage [Buhan *et al.*, 2021]. Last but not the least, if the leakage evaluation indicates that the device does not meet the required side-channel resistance, such as that specified by the Common Criteria standard (ISO/IEC 15408), then there is no option but to discard the device and re-fabricate it after incorporating appropriate countermeasures in the design, which can be extremely expensive.

3.2 Pre-silicon Techniques

Pre-silicon techniques perform the evaluation of power side-channel vulnerability in the early design stages of a device. They typically involve behavioural or functional simulations of the design, at the RTL or gate-level, combined with an analysis of the design specifications of the device, in order to obtain a proxy for the power traces. This is followed by applying power models (such as Hamming weight and Hamming distance) and suitable evaluation metrics. For instance, tools such as RTL-PSC [He *et al.*, 2019], Karna [Slpsk *et al.*, 2019] and CASCADE [Šijačić

et al., 2020] can be used to assess the side-channel leakage of cryptographic designs such as AES, at the RTL or gate-level netlist stage. Power attack Leakage ANalyzer (PLAN) [KF *et al.*, 2020] has been used to evaluate the power side-channel leakage scores of the Shakti-C processor design at the post-synthesis netlist stage, using behavioural simulations, Hamming-weight based power models and correlation analysis. The work by Zoni et. al. [Zoni *et al.*, 2018] analyses the power side-channel vulnerability of an open-source RISC CPU microarchitecture at the gate-level netlist stage. SCRIPT [Nahiyan *et al.*, 2020] is an automated CAD framework which has been used to evaluate AES designs as well as a multiplication unit of a RISC processor at gate-level, using information-flow tracking techniques.

Pre-silicon vulnerability estimation techniques are performed earlier in the development process as compared to post-silicon techniques. So, they provide flexibility to the hardware designer to evaluate a design, identify potential vulnerabilities, and apply targeted countermeasures before fabrication. While pre-silicon techniques give an early estimate of power side-channel vulnerability, they may not be as accurate as post-silicon techniques since the design stage may not capture physical details such as layout, interactions between neighbouring wires and so on, which also contribute to side-channel leakages. Moreover, many of these techniques also rely on statistical analysis, which require a large number of simulations, which is extremely time-consuming. For instance, PLAN requires almost a month to perform the required number of simulations to evaluate the side-channel vulnerability of the Shakti-C processor [KF *et al.*, 2020].

CHAPTER 4

PLAN: POWER ATTACK LEAKAGE ANALYZER

PLAN [KF *et al.*, 2020] is one of the recent pre-silicon power side-channel vulnerability evaluation tools. PLAN takes as input a Verilog RTL source code or gate-level netlist of a hardware design, along with a reference or *oracle* signal, and evaluates fine-grained power side-channel leakage scores (with respect to the oracle signal) for each signal/module in the design, by following three main steps:

4.1 Behavioural Simulation

Firstly, PLAN performs a large number (say N) of behavioural simulations on the input design. During each simulation, it collects the values taken by each signal in the design, as a proxy for power traces.

Let us consider two such signals of interest, the ***oracle*** or reference signal, and another signal ***sig*** for which we would like to compute the power side-channel leakage score with respect to the *oracle*. For example, the *oracle* could be the value of the `XOR` operation between a plain text and a secret key, where the plain text bit varies in each simulation and the secret key remains constant, while the other signal *sig* could be any signal in one of the subsequent cipher operations.

The ***oracle trace*** is the list of values taken by the oracle signal during each simulation:

$$oracle = [o_1, o_2 \cdots o_N]$$

The ***side-channel trace*** is the list of values taken by the signal *sig* during each simulation:

$$sig = [s_1, s_2 \cdots s_N]$$

PLAN collects such a side-channel trace is collected for each signal in the design, and for each clock cycle of operation. However, we shall look at only one such signal *sig*, and for only one clock cycle, in the subsequent analysis. The same analysis can be extended to all other signals in the design, and for multiple clock cycles. Please refer to [KF *et al.*, 2020] for more details.

4.2 Pattern Extraction

The oracle and side-channel traces obtained from the behavioural simulations are further analysed by extracting pairwise distance patterns in these traces, by applying the Hamming distance metric.

The ***Hamming distance*** (HD) between two binary strings A, B of equal length is defined as the total number of positions in which A and B differ, i.e.,

$$\texttt{HD}(A, B) = \sum_i A_i \oplus B_i$$

Let us denote the list of $\binom{N}{2}$ pairs of simulation indices taken two at a time as follows:

$$pairs = [(1, 2), (1, 3), \cdots (N-1, N)]$$

where $pairs[i] = (i_1, i_2)$.

For pattern extraction, PLAN calculates the distance vectors (of length $\binom{N}{2}$) for

both the oracle trace as well as the side-channel trace as follows:

$$x = [\texttt{HD}(o_{i_1}, o_{i_2})]\ \forall (i_1, i_2) \in \textit{pairs}$$

$$y = [\texttt{HD}(s_{i_1}, s_{i_2})]\ \forall (i_1, i_2) \in \textit{pairs}$$

i.e., $x_i = \texttt{HD}(o_{i_1}, o_{i_2})$, and $y_i = \texttt{HD}(s_{i_1}, s_{i_2})$.

4.3 Correlation Analysis

The extracted patterns are statistically analysed using a metric known as ***Side-channel Vulnerability Factor*** (SVF) [Demme *et al.*, 2012], which is based on Pearson's correlation, to obtain the signal-wise/module-wise leakage scores with respect to the oracle signal.

$$L(sig) = \frac{\sum_i (x_i - \overline{x}) \cdot (y_i - \overline{y})}{\sqrt{\sum_i (x_i - \overline{x})^2} \cdot \sqrt{\sum_i (y_i - \overline{y})^2}} \tag{4.1}$$

where $\overline{x}, \overline{y}$ represent the mean of the vectors x, y respectively.

CHAPTER 5

FORMAL ANALYSIS OF LEAKAGE SCORES

5.1 Assumptions

We make the following assumptions in our analysis:

1. The oracle signal, *oracle*, is a 1-bit signal, so the oracle trace $[o_1 \cdots o_N]$ consists of only 1-bit values, which can be either 0 or 1.
2. The signal, *sig*, is also a 1-bit signal, so the side-channel trace $[s_1 \cdots s_N]$ also consists of only 1-bit values, which can be either 0 or 1.
3. The distance vector x is constructed by taking the Hamming distance between every pair of values in the oracle trace, which consists of only 0s and 1s. So, even the distance vector x contains only 0s and 1s.
4. The distance vector y is constructed by taking the Hamming distance between every pair of values in the side-channel trace, which consists of only 0s and 1s. So, even the distance vector y contains only 0s and 1s.
5. The oracle trace $[o_1 \cdots o_N]$ has almost equal number of 0s and 1s, when N is sufficiently large. This is because the oracle is usually the result of an `XOR` operation between the random input in each simulation and the fixed secret key, so the oracle trace also would have the same random distribution as the input. Thus, we have:

$$\left|\{i \mid o_i = 0\}\right| = \frac{N}{2}, \quad \left|\{i \mid o_i = 1\}\right| = \frac{N}{2}$$

5.2 Analysis of Leakage Formula

Based on the above assumptions, we formally analyse the power side-channel leakage score formula given by Equation 4.1.

For the distance vector x,

$$\begin{aligned}
\left|\{i \mid x_i = 0\}\right| &= \left|\{(i_1, i_2) \mid \texttt{HD}(o_{i_1}, o_{i_2}) = 0\}\right| \\
&= \left|\{(i_1, i_2) \mid o_{i_1} = o_{i_2}\}\right| \\
&= \left|\{(i_1, i_2) \mid o_{i_1} = o_{i_2} = 0\}\right| + \left|\{(i_1, i_2) \mid o_{i_1} = o_{i_2} = 1\}\right| \\
&= \binom{N/2}{2} + \binom{N/2}{2} \\
&= \frac{N}{2}\left(\frac{N}{2} - 1\right) \\
&= \frac{N^2 - 2N}{4} \\
\left|\{i \mid x_i = 1\}\right| &= \binom{N}{2} - \left|\{i \mid x_i = 0\}\right| \\
&= \frac{N(N-1)}{2} - \left(\frac{N^2 - 2N}{4}\right) \\
&= \frac{N^2}{4}
\end{aligned}$$

Hence, the mean $\overline{x}$ is:

$$\begin{aligned}
\overline{x} = \frac{\sum_i x_i}{\binom{N}{2}} = \frac{\left|\{i \mid x_i = 1\}\right|}{\binom{N}{2}} = \frac{N^2/4}{N(N-1)/2} &= \frac{1}{2(1 - \frac{1}{N})} \\
\approx \frac{1}{2} \quad \text{for sufficiently large } N &
\end{aligned}$$

For the distance vector y, the mean $\overline{y}$ is:

$$\overline{y} = \frac{\sum_i y_i}{\binom{N}{2}}$$

The numerator of Equation 4.1 simplifies to:

$$\begin{aligned}
\sum_i (x_i - \overline{x}) \cdot (y_i - \overline{y}) &= \sum_{i|x_i=0} -\frac{1}{2}(y_i - \overline{y}) + \sum_{i|x_i=1} \frac{1}{2}(y_i - \overline{y}) \\
&= \frac{1}{2} \cdot \left(\sum_{i|x_i=1} y_i - \sum_{i|x_i=1} \overline{y} - \sum_{i|x_i=0} y_i + \sum_{i|x_i=0} \overline{y} \right) \\
&= \frac{1}{2} \cdot \left(\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i + \overline{y} \cdot \left(\left| \{i \,|\, x_i = 0\} \right| - \left| \{i \,|\, x_i = 1\} \right| \right) \right) \\
&= \frac{1}{2} \cdot \left(\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i + \overline{y} \cdot \left(\frac{N^2 - 2N}{4} - \frac{N^2}{4} \right) \right) \\
&= \frac{1}{2} \cdot \left(\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y} \cdot \frac{N}{2} \right)
\end{aligned}$$

The denominator of Equation 4.1 has two components, which can be simplified as follows:

$$\begin{aligned}
\sum_i (x_i - \overline{x})^2 &= \sum_{i|x_i=0} \left(0 - \frac{1}{2}\right)^2 + \sum_{i|x_i=1} \left(1 - \frac{1}{2}\right)^2 \\
&= \sum_{i|x_i=0} \left(-\frac{1}{2}\right)^2 + \sum_{i|x_i=1} \left(\frac{1}{2}\right)^2 \\
&= \frac{1}{4} \cdot \left(\left| \{i \,|\, x_i = 0\} \right| + \left| \{i \,|\, x_i = 1\} \right| \right) \\
&= \frac{1}{4} \cdot \binom{N}{2} \\
\sum_i (y_i - \overline{y})^2 &= \sum_i y_i^2 + \binom{N}{2} \cdot \overline{y}^2 - 2 \cdot \overline{y} \cdot \sum_i y_i \\
&= \sum_i y_i^2 + \binom{N}{2} \cdot \overline{y}^2 - 2 \cdot \binom{N}{2} \cdot \overline{y}^2 \\
&= \sum_i y_i - \binom{N}{2} \cdot \overline{y}^2 \quad \text{since } y_i^2 = y_i \\
&= \binom{N}{2} \cdot \overline{y} - \binom{N}{2} \cdot \overline{y}^2 \\
&= \binom{N}{2} \cdot \overline{y} \cdot (1 - \overline{y})
\end{aligned}$$

Hence, Equation 4.1 simplifies to:

$$
\begin{aligned}
L(sig) &= \frac{\frac{1}{2}\Big(\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y}\cdot\frac{N}{2}\Big)}{\sqrt{\frac{1}{4}\binom{N}{2}}\cdot\sqrt{\binom{N}{2}\cdot\overline{y}\cdot(1-\overline{y})}} \\
&= \frac{\frac{1}{2}\Big(\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y}\cdot\frac{N}{2}\Big)}{\frac{1}{2}\binom{N}{2}\cdot\sqrt{\overline{y}\cdot(1-\overline{y})}} \\
&= \frac{\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y}\cdot\frac{N}{2}}{\binom{N}{2}\cdot\sqrt{\overline{y}\cdot(1-\overline{y})}}
\end{aligned}
$$

Thus, we have:

$$
L(sig) = \frac{\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y}\cdot\frac{N}{2}}{\binom{N}{2}\cdot\sqrt{\overline{y}\cdot(1-\overline{y})}} \tag{5.1}
$$

We shall now express each component of Equation 5.1 in terms of signal probability and conditional signal probabilities:

$$SP = \Pr(sig = 1)$$

$$SP_0 = \Pr(sig = 1 | oracle = 0)$$

$$SP_1 = \Pr(sig = 1 | oracle = 1)$$

Since the oracle trace has $\frac{N}{2}$ 0s and $\frac{N}{2}$ 1s,

$$\Pr(oracle = 0) = \Pr(oracle = 1) = \frac{N/2}{N} = \frac{1}{2}$$

Using the formula of conditional probability[1]:

$$\begin{aligned}\Pr(sig = 1 \cap oracle = 0) &= \Pr(sig = 1 \,|\, oracle = 0) \cdot \Pr(oracle = 0) \\ &= SP_0 \cdot \frac{1}{2} = \frac{SP_0}{2} \\ \Pr(sig = 1 \cap oracle = 1) &= \Pr(sig = 1 \,|\, oracle = 1) \cdot \Pr(oracle = 1) \\ &= SP_1 \cdot \frac{1}{2} = \frac{SP_1}{2}\end{aligned}$$

By applying the total probability theorem[2]:

$$\begin{aligned}\Pr(sig = 0 \cap oracle = 0) &= \Pr(oracle = 0) - \Pr(sig = 1 \cap oracle = 0) \\ &= \frac{1}{2} - \frac{SP_0}{2} = \frac{1 - SP_0}{2} \\ \Pr(sig = 0 \cap oracle = 1) &= \Pr(oracle = 1) - \Pr(sig = 1 \cap oracle = 1) \\ &= \frac{1}{2} - \frac{SP_1}{2} = \frac{1 - SP_1}{2}\end{aligned}$$

Again, by applying the total probability theorem,

$$\begin{aligned}SP &= \Pr(sig = 1) \\ &= \Pr(sig = 1 \cap oracle = 0) + \Pr(sig = 1 \cap oracle = 1) \\ &= \frac{SP_0}{2} + \frac{SP_1}{2} = \frac{SP_0 + SP_1}{2}\end{aligned}$$

[1]Conditional Probability Formula: $\Pr(A \cap B) = \Pr(A|B) \cdot \Pr(B)$

[2]Total Probability Theorem: $\Pr(A) = \Pr(A \cap B) + \Pr(A \cap \overline{B})$

Now, we express each component of Equation 5.1 in terms of SP, SP_0, SP_1:

$$\sum_{i|x_i=1} y_i = \left|\{i \mid y_i = 1, x_i = 1\}\right| = \left|\{(i_1, i_2) \mid s_{i_1} \neq s_{i_2}, o_{i_1} \neq o_{i_2}\}\right|$$

$$= \left|\{(i_1, i_2) \mid s_{i_1} = 0, s_{i_2} = 1, o_{i_1} = 0, o_{i_2} = 1\}\right|$$
$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 0, s_{i_2} = 1, o_{i_1} = 1, o_{i_2} = 0\}\right|$$
$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 1, s_{i_2} = 0, o_{i_1} = 0, o_{i_2} = 1\}\right|$$
$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 1, s_{i_2} = 0, o_{i_1} = 1, o_{i_2} = 0\}\right|$$

$$= \binom{N}{2} \cdot \Big[\Pr(s_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_1} = 0 \cap o_{i_2} = 1)$$
$$+ \Pr(s_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_1} = 1 \cap o_{i_2} = 0)$$
$$+ \Pr(s_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_1} = 0 \cap o_{i_2} = 1)$$
$$+ \Pr(s_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_1} = 1 \cap o_{i_2} = 0)\Big]$$

$$= \binom{N}{2} \cdot \Big[\Pr(s_{i_1} = 0 \cap o_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_2} = 1)$$
$$+ \Pr(s_{i_1} = 0 \cap o_{i_1} = 1 \cap s_{i_2} = 1 \cap o_{i_2} = 0)$$
$$+ \Pr(s_{i_1} = 1 \cap o_{i_1} = 0 \cap s_{i_2} = 0 \cap o_{i_2} = 1)$$
$$+ \Pr(s_{i_1} = 1 \cap o_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_2} = 0)\Big]$$

$$= \binom{N}{2} \cdot \Big[\Pr(sig = 0 \cap oracle = 0) \cdot \Pr(sig = 1 \cap oracle = 1)$$
$$+ \Pr(sig = 0 \cap oracle = 1) \cdot \Pr(sig = 1 \cap oracle = 0)$$
$$+ \Pr(sig = 1 \cap oracle = 0) \cdot \Pr(sig = 0 \cap oracle = 1)$$
$$+ \Pr(sig = 1 \cap oracle = 1) \cdot \Pr(sig = 0 \cap oracle = 0)\Big]$$

since i_1, i_2 are independent

$$= \binom{N}{2}\left[\frac{(1 - SP_0)}{2} \cdot \frac{SP_1}{2} + \frac{(1 - SP_1)}{2} \cdot \frac{SP_0}{2} + \frac{SP_1}{2} \cdot \frac{(1 - SP_0)}{2} + \frac{SP_0}{2} \cdot \frac{(1 - SP_1)}{2}\right]$$

$$= \frac{1}{2}\binom{N}{2}\Big[SP_0 + SP_1 - 2 \cdot SP_0 \cdot SP_1\Big]$$

$$\sum_{i|x_i=0} y_i = \left|\{i \mid y_i = 1, x_i = 0\}\right| = \left|\{(i_1, i_2) \mid s_{i_1} \neq s_{i_2}, o_{i_1} = o_{i_2}\}\right|$$

$$= \left|\{(i_1, i_2) \mid s_{i_1} = 0, s_{i_2} = 1, o_{i_1} = 0, o_{i_2} = 0\}\right|$$

$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 0, s_{i_2} = 1, o_{i_1} = 1, o_{i_2} = 1\}\right|$$

$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 1, s_{i_2} = 0, o_{i_1} = 0, o_{i_2} = 0\}\right|$$

$$+ \left|\{(i_1, i_2) \mid s_{i_1} = 1, s_{i_2} = 0, o_{i_1} = 1, o_{i_2} = 1\}\right|$$

$$= \binom{N}{2} \cdot \Big[\Pr(s_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_1} = 0 \cap o_{i_2} = 0)$$

$$+ \Pr(s_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_1} = 1 \cap o_{i_2} = 1)$$

$$+ \Pr(s_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_1} = 0 \cap o_{i_2} = 0)$$

$$+ \Pr(s_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_1} = 1 \cap o_{i_2} = 1)\Big]$$

$$= \binom{N}{2} \cdot \Big[\Pr(s_{i_1} = 0 \cap o_{i_1} = 0 \cap s_{i_2} = 1 \cap o_{i_2} = 0)$$

$$+ \Pr(s_{i_1} = 0 \cap o_{i_1} = 1 \cap s_{i_2} = 1 \cap o_{i_2} = 1)$$

$$+ \Pr(s_{i_1} = 1 \cap o_{i_1} = 0 \cap s_{i_2} = 0 \cap o_{i_2} = 0)$$

$$+ \Pr(s_{i_1} = 1 \cap o_{i_1} = 1 \cap s_{i_2} = 0 \cap o_{i_2} = 1)\Big]$$

$$= \binom{N}{2} \cdot \Big[\Pr(sig = 0 \cap oracle = 0) \cdot \Pr(sig = 1 \cap oracle = 0)$$

$$+ \Pr(sig = 0 \cap oracle = 1) \cdot \Pr(sig = 1 \cap oracle = 1)$$

$$+ \Pr(sig = 1 \cap oracle = 0) \cdot \Pr(sig = 0 \cap oracle = 0)$$

$$+ \Pr(sig = 1 \cap oracle = 1) \cdot \Pr(sig = 0 \cap oracle = 1)\Big]$$

since i_1, i_2 are independent

$$= \binom{N}{2}\left[\frac{(1 - SP_0)}{2} \cdot \frac{SP_0}{2} + \frac{(1 - SP_1)}{2} \cdot \frac{SP_1}{2} + \frac{SP_0}{2} \cdot \frac{(1 - SP_0)}{2} + \frac{SP_1}{2} \cdot \frac{(1 - SP_1)}{2}\right]$$

$$= \frac{1}{2}\binom{N}{2}\left[SP_0 + SP_1 - SP_0^2 - SP_1^2\right]$$

$$
\begin{aligned}
\overline{y} &= \frac{\sum_i y_i}{\binom{N}{2}} = \frac{\left|\{i|y_i = 1\}\right|}{\binom{N}{2}} \\
&= \frac{\left|\{i|y_i = 1, x_i = 1\}\right| + \left|\{i|y_i = 1, x_i = 1\}\right|}{\binom{N}{2}} \\
&= \frac{\frac{1}{2}\binom{N}{2}\left[SP_0 + SP_1 - SP_0^2 - SP_1^2 + SP_0 + SP_1 - 2 \cdot SP_0 \cdot SP_1\right]}{\binom{N}{2}} \\
&= \frac{1}{2} \cdot \left[2 \cdot (SP_0 + SP_1) - (SP_0 + SP_1)^2\right] \\
&= \frac{1}{2} \cdot \left[2 \cdot (2SP) - (2SP)^2\right] \\
&= \frac{1}{2} \cdot \left[4SP - 4SP^2\right] \\
&= 2 \cdot SP \cdot (1 - SP)
\end{aligned}
$$

From the above results, we can show that the numerator of Equation 5.1 simplifies to

$$
\begin{aligned}
&\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y} \cdot \frac{N}{2} \\
&\quad = \frac{1}{2}\binom{N}{2}\left[SP_0 + SP_1 - 2 \cdot SP_0 \cdot SP_1 - SP_0 - SP_1 + SP_0^2 + SP_1^2\right] - \frac{N}{2} \cdot \overline{y} \\
&\quad = \frac{1}{2}\binom{N}{2}(SP_1 - SP_0)^2 - \frac{N}{2} \cdot \overline{y}
\end{aligned}
$$

Thus, Equation 5.1 can be simplified to:

$$
\begin{aligned}
L(sig) &= \frac{\sum_{i|x_i=1} y_i - \sum_{i|x_i=0} y_i - \overline{y} \cdot \frac{N}{2}}{\binom{N}{2} \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} \\
&= \frac{\frac{1}{2}\binom{N}{2}(SP_1 - SP_0)^2 - \frac{N}{2} \cdot \overline{y}}{\binom{N}{2} \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} \\
&= \frac{(SP_1 - SP_0)^2}{2 \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} - \frac{N \cdot \overline{y}}{2 \cdot \binom{N}{2} \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} \\
&= \frac{(SP_1 - SP_0)^2}{2 \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} - \frac{1}{(N-1) \cdot \sqrt{\frac{1}{\overline{y}} - 1}} \\
&\approx \frac{(SP_1 - SP_0)^2}{2 \cdot \sqrt{\overline{y} \cdot (1-\overline{y})}} \quad \text{where } \overline{y} = 2 \cdot SP \cdot (1 - SP)
\end{aligned}
$$

where we have approximated the second term to 0 (since N is large, and $\overline{y} < 1 \implies \frac{1}{\overline{y}} > 1$, so the denominator of the second term is very large, and hence it can be approximated to 0).

In this way, we see that the power side-channel leakage score of a signal with respect to a reference signal in a design can be expressed in terms of the signal probability and conditional signal probabilities of the signal with respect to the reference signal.

5.3 Leakage Formulae

Based on our assumptions in Section 5.1, we obtain the above result which is stated by the following theorem:

Theorem 1. *Suppose the reference signal is a* 1*-bit signal A, then the leakage* L(*sig*) *of any*

1-bit signal sig in the circuit is:

$$L(sig) = \frac{\left[SP_{A=1}(sig) - SP_{A=0}(sig)\right]^2}{2 \cdot \sqrt{D \cdot (1 - D)}} \tag{5.2}$$

where $D = 2 \cdot SP(sig) \cdot (1 - SP(sig))$.

A general digital circuit design is usually composed of both single-bit as well as multi-bit signals. For example, an 8-bit full adder design would have two 8-bit values and a 1-bit carry as input, and an 8-bit sum along with a 1-bit carry as output. So, in the following analysis, we relax the assumptions of Section 5.1 to deal with both multi-bit signals as well as multi-bit reference signals.

Let us try to understand this using a simple example of a 4-bit signal $B[3:0]$ (with a 1-bit reference signal A). Suppose we apply the leakage formula as per Equation 5.2 and calculate the leakage scores of each bit of B, i.e., $L(B[0])$, $L(B[1])$, $L(B[2])$, $L(B[3])$. To calculate the leakage score of the 4-bit signal B as a whole, we might initially propose adding up the individual leakage scores. However, looking at the quadratic relation in the leakage formula of Equation 5.2, we infer that the leakage scores follow vectorial addition (the leakage scores of the individual bits are comparable to the magnitudes of the orthogonal projections of a vector). This result is stated formally in the following theorem:

Theorem 2. *Suppose the leakage scores of the individual bits of a w-bit signal sig are* $L(sig[w-1]), L(sig[w-2]) \cdots L(sig[0])$, *then the total leakage score of the signal sig is:*

$$L(sig) = \sqrt{\sum_{i=0}^{w-1} L(sig[i])^2} \tag{5.3}$$

Similarly, given the aggregated leakage score of an n-bit signal B, we can assume

that the leakage is equally split among its individual bits, similar to the magnitude of the orthogonal components of a normalized n-dimensional vector with equal projections on all axes.

Theorem 3. *Suppose the leakage score of a w-bit signal sig is L(sig), then the leakage score of each individual bit sig[i] (i = w − 1 ··· 0) is:*

$$L(sig[i]) = \frac{L(sig)}{\sqrt{w}} \tag{5.4}$$

Suppose the reference signal A is an m-bit signal. To calculate the leakage score of a 1-bit signal *sig*, we need to measure the leakage of *sig* with respect to each bit of A, and combine these leakages by applying the root mean square (RMS) operation. Further, if *sig* also is a multi-bit signal, we would have to combine the leakage scores of each bit of *sig*. The following theorems state the mathematical formulae to be used for such cases.

Theorem 4. *Suppose the reference signal is an m-bit signal A, then the leakage score of any 1-bit signal sig is:*

$$L(sig) = \sqrt{\frac{1}{m} \cdot \sum_{j=0}^{m-1} \frac{\left[SP_{A[j]=1}(sig) - SP_{A[j]=0}(sig)\right]^2}{2 \cdot \sqrt{D \cdot (1-D)}}} \tag{5.5}$$

where $D = 2 \cdot SP(sig) \cdot (1 - SP(sig))$.

Theorem 5. *Suppose the reference signal is an m-bit signal A, then the leakage score of*

any w-bit signal sig is:

$$
\begin{aligned}
L(sig) &= \sqrt{\sum_{i=0}^{w-1} L(sig[i])^2} \\
&= \sqrt{\sum_{i=0}^{w-1} \frac{1}{m} \cdot \sum_{i=0}^{m-1} \frac{\left[SP_{A[j]=1}(sig) - SP_{A[j]=0}(sig)\right]^2}{2\sqrt{D \cdot (1-D)}}}
\end{aligned}
\tag{5.6}
$$

where $D = 2 \cdot SP(sig) \cdot (1 - SP(sig))$.

We have experimentally confirmed the correctness of the formulae corresponding to Equations 5.3 through 5.6 using existing leakage score evaluation tools such as PLAN.

CHAPTER 6

INSTANT LEAKAGE ANALYSIS

We have developed a tool which takes as inputs:

1. a Verilog digital circuit design: either RTL code or gate-level netlist [1]
2. the standard cell module definitions used in the above design
3. a list of reference signals (along with their corresponding values) in the design
4. a reference signal in the design

and identifies the signal-wise power side-channel leakage scores of each signal in the design, with respect to the reference signal. Our tool comprises 4 modules[2], as illustrated in Figure 6.1. The role of each module is explained in detail in the following subsections, taking the example of the sample circuit (Figure 6.2).

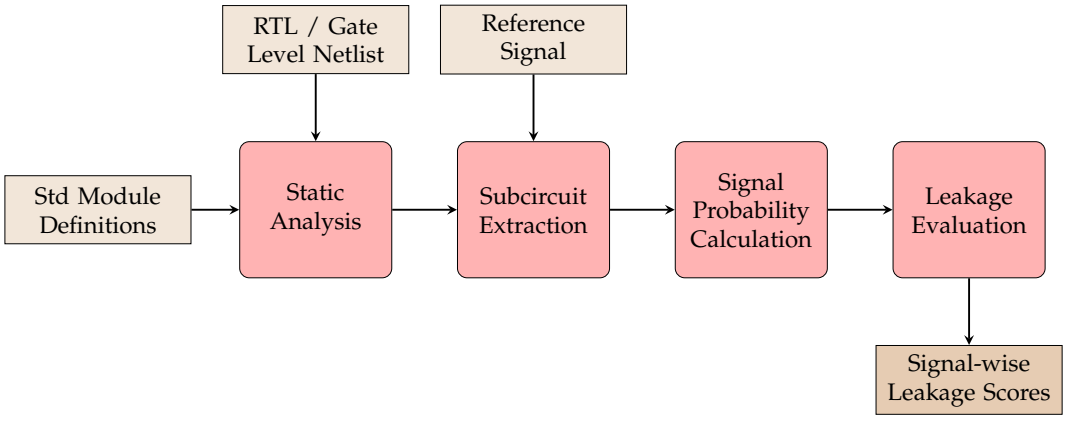


Figure 6.1: Overview of Tool for Signal-wise Leakage Scores Calculation

[1] synthesised using the Synopsys Design Compiler Version O-2018.06-SP5-4 for linux64
[2] implemented in Python; code repository: `https://github.com/avlakshmy/ddp-psca-code`

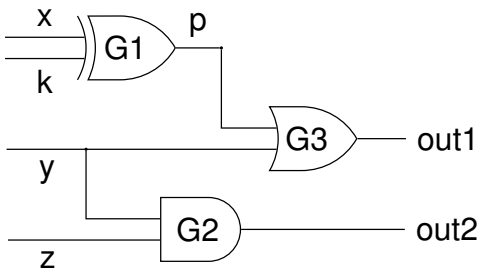


Figure 6.2: Sample Circuit

6.1 Static Analysis

The first module in our tool takes as input the Verilog digital circuit design to be analysed (either RTL or gate-level netlist), along with the module definitions of the standard cells instantiated in the design. These inputs are parsed using the Pyverilog [Takamaeda-Yamazaki, 2015] library to obtain a directed graph representation, where the nodes represent the logic gates in the design while the edges describe the signals. Each edge or signal is labelled with its corresponding logical expression. We use the following syntax to represent the logical expressions:

- The logical expression of a primary input signal is the same as the signal name itself.
- The logical expression of other signals is expressed in terms of the logic gate that drives them, and the input signals to that logic gate:

$$[gate, input1, input2 \cdots]$$

For example, the logical expressions of the signals in the example circuit are shown in Table 6.1.

Table 6.1: Logical Expressions in the Sample Circuit

Signal	Logical Expression
x	x
k	k
y	y
z	z
p	[XOR, x, k]
out1	[OR, p, y]
out2	[AND, y, z]

6.2 Subcircuit Extraction

The second module in our tool takes as input the directed graph representation of the circuit, along with the reference signal, and traces the path of the reference signal through the design, to identify the portions of the design which operate on the reference signal. This involves two components:

1. Extracting a subgraph of the directed graph representation composed of nodes and edges which are reachable from the reference signal
2. Identifying the other inputs feeding into this subgraph, apart from the reference signal

We perform this by tracing the path of the reference signal in the directed graph representation of the design, using a method similar to a breadth-first traversal, while simultaneously collecting the other required inputs. This is described in Algorithm 1.

In the case of the example circuit of Figure 6.2, let us consider the signal p to be the reference signal. When we trace the path of p in the directed graph, we reach the OR gate, G3, followed by the output of the gate, out1. The other input feeding into this subgraph is y. The extracted subcircuit is shown in Figure 6.3.

Algorithm 1 Subcircuit Extraction

```
1: function EXTRACTSUBCIRCUIT(C, A_1 ··· A_m)
2:     Let C be a digital circuit design.
3:     Let A_1 ··· A_m be the reference signals.
4:     Let instList be the set of instances in the top module of C.
5:     Let signalNames be an empty list.
6:     Let forward ← [A_1 ··· A_m]
7:     while forward is not empty do
8:         Let forwardTemp be an empty set.
9:         for sig ∈ forward do
10:            for inst ∈ instList do
11:                if sig ∈ inputs of inst then
12:                    Add inputs, internal signals, outputs of inst to signalNames.
13:                    Add outputs of inst to forwardTemp.
14:                end if
15:            end for
16:        end for
17:        forward ← forwardTemp
18:    end while
19: end function
```

The forward tracing of the reference signal p is shown in red, while the other input y feeding into the subcircuit is shown in blue.

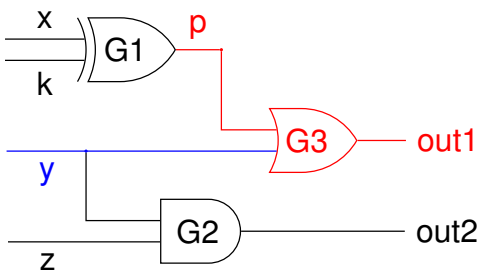


Figure 6.3: Extracted Subcircuit

6.3 Signal Probability Calculation

In the third module, we calculate the signal probabilities as well as conditional signal probabilities of each signal in the extracted subcircuit, with respect to the reference signal. The approach for signal probability calculation is described in

Algorithm 3. It calculates the signal probability and conditional signal probability values of a signal *sig* by recursively calculating the corresponding values of the signals in its logical expression, and then applying the corresponding incremental signal probability formula (Algorithm 2). The base cases in the recursion are the inputs to the design and the reference signal bits.

- The signal probability of all the primary inputs as well as the reference signal bits is assigned to be 0.5.
- The conditional signal probabilities of each of the primary inputs with respect to each of the reference signal bits taking the value 0/1 are assigned to be 0.5 each.
- The conditional signal probabilities of the reference signal bits with respect to each of the reference signal bits taking the value 0/1 are assigned to be 0 and 1 respectively.

Algorithm 2 Incremental Signal Probability

```
1:  function incSignalProbability(gate, s[])
2:      Let gate be a logic gate with input signal(s) A (, B).
3:      Let s[] be the mapping from signals to signal probabilities.
4:      if gate == NOT then
5:          return 1 − s[A]
6:      end if
7:      if gate == AND then
8:          return s[A] · s[B]
9:      end if
10:     if gate == OR then
11:         return s[A] + s[B] − s[A] · s[B]
12:     end if
13:     if gate == XOR then
14:         return s[A] + s[B] − 2 · s[A] · s[B]
15:     end if
16: end function
```

The signal probability and conditional signal probability calculations of the signal out1 in the extracted subcircuit (Figure 6.3) is as follows:

Algorithm 3 Signal Probability Calculation

1: **function** signalProbability(C, sig, $A_1 \cdots A_m$, $I_1 \cdots I_n$, $truthTableMap$, $\widehat{s}$, $\widehat{s_0}$, $\widehat{s_1}$)
2: Let C be a digital circuit design.
3: Let sig be a signal in C.
4: Let $A_1 \cdots A_m$ be the reference signal bits in C.
5: Let $I_1 \cdots I_n$ be the input signal bits in C.
6: Let $truthTableMap$ be the mapping from the signals in C to their corresponding logical expressions.
7: Let $\widehat{s}$ be the mapping from signals in C to corresponding signal probabilities.
8: Let $\widehat{s_0}[], \widehat{s_1}[]$ be mappings from signals in C to their corresponding list of m conditional signal probabilities, given each reference signal $A_j = 0, 1$.
9: **if** $sig \in \widehat{s}$ **then return**
10: **end if**
11: Let $[gate, A, B] \leftarrow truthTableMap[sig]$
12: signalProbability($A, \widehat{s}, \widehat{s_0}, \widehat{s_1}, C$)
13: signalProbability($B, \widehat{s}, \widehat{s_0}, \widehat{s_1}, C$)
14: $\widehat{s}[sig] \leftarrow$ incSignalProbability($gate, \widehat{s}$)
15: **for** $i = 1, 2 \cdots m$ **do**
16: $\widehat{s_0}[sig][A_i] \leftarrow$ incSignalProbability($gate, \widehat{s_0}$)
17: $\widehat{s_1}[sig][A_i] \leftarrow$ incSignalProbability($gate, \widehat{s_1}$)
18: **end for**
19: **end function**

$$
\begin{aligned}
SP(\mathsf{out1}) &= SP([\mathtt{OR}, \mathsf{p}, \mathsf{y}]) \\
&= SP(\mathsf{p}) + SP(\mathsf{y}) - SP(\mathsf{p}) \cdot SP(\mathsf{y}) \\
&= 0.5 + 0.5 - 0.5 \cdot 0.5 = 0.75 \\
SP_{\mathsf{p}=0}(\mathsf{out1}) &= SP_{\mathsf{p}=0}([\mathtt{OR}, \mathsf{p}, \mathsf{y}]) \\
&= SP_{\mathsf{p}=0}(\mathsf{p}) + SP_{\mathsf{p}=0}(\mathsf{y}) - SP_{\mathsf{p}=0}(\mathsf{p}) \cdot SP_{\mathsf{p}=0}(\mathsf{y}) \\
&= 1 + 0.5 - 1 \cdot 0.5 = 1 \\
SP_{\mathsf{p}=1}(\mathsf{out1}) &= SP_{\mathsf{p}=1}([\mathtt{OR}, \mathsf{p}, \mathsf{y}]) \\
&= SP_{\mathsf{p}=1}(\mathsf{p}) + SP_{\mathsf{p}=1}(\mathsf{y}) - SP_{\mathsf{p}=1}(\mathsf{p}) \cdot SP_{\mathsf{p}=1}(\mathsf{y}) \\
&= 0 + 0.5 - 0 \cdot 0.5 = 0.5
\end{aligned}
$$

The signal probability and conditional signal probabilities of the signals in the extracted subcircuit are shown in Table 6.2.

Table 6.2: Signal Probability Calculation in Extracted Subcircuit

sig	$SP(sig)$	$SP_{\mathsf{p}=0}(sig)$	$SP_{\mathsf{p}=1}(sig)$
y	0.5	0.5	0.5
p	0.5	1.0	0.0
out1	0.75	1.0	0.5

6.4 Leakage Evaluation

In the final module of our tool, we calculate the signal-wise power side-channel leakage scores for each signal in the extracted subgraph using the formulae described in Chapter 5. In case of the sample circuit, since all the signals as well as the reference signal are 1-bit wide, the corresponding formula is:

$$L(sig) = \frac{[SP_{\mathsf{p}=1}(sig) - SP_{\mathsf{p}=0}(sig)]^2}{2 \cdot \sqrt{D \cdot (1 - D)}}$$

where $D = 2 \cdot SP(sig) \cdot (1 - SP(sig))$.

The leakage scores of the signals in the example circuit are shown in Table 6.3.

Table 6.3: Leakage Score Calculation in Example Circuit

sig	$SP(sig)$	$SP_{\mathsf{p}=0}(sig)$	$SP_{\mathsf{p}=1}(sig)$	$L(sig)$
y	0.5	0.5	0.5	0.00
p	0.5	1.0	0.0	1.00
out1	0.75	1.0	0.5	0.26

CHAPTER 7

RESULTS

7.1 Evaluation of Small Designs

We have applied our tool to evaluate the signal-wise power side-channel leakage scores for the following digital circuit designs (Verilog RTL/ gate-level netlists):

1. 2-output circuit of `NAND` gates (c17) [Hansen *et al.*, 1999]
2. 2-bit Full Adder (FA-2)
3. 4-bit Full Adder (FA-4)
4. 8-bit Full Adder (FA-8)
5. 27-channel interrupt controller (c432) [Hansen *et al.*, 1999]
6. PRESENT cipher (1 round of encryption) (PRE-ENC-1R) [Bogdanov *et al.*, 2007]
7. PRESENT cipher (1 round of decryption) (PRE-DEC-1R) [Bogdanov *et al.*, 2007]
8. PRESENT cipher (2 rounds of encryption) (PRE-ENC-2R) [Bogdanov *et al.*, 2007]
9. PRESENT cipher (2 rounds of decryption) (PRE-DEC-2R) [Bogdanov *et al.*, 2007]

In order to validate the results of our tool, we have compared the leakage scores given by our tool with those given by PLAN [KF *et al.*, 2020], based on the following metrics:

1. **Time taken** by PLAN as well as our tool to calculate the signal-wise leakage scores of the entire digital circuit design

Table 7.1: Validation: Comparison of Our Tool with PLAN

Design	PLAN		Our Tool		# Common Signals	Pearson's Correlation	Spearman's Correlation
	# Signals	Time Taken	# Signals	Time Taken			
c17	11	~ 1.6 mins	19	0.88 s	7	0.975	0.923
FA-2	30	~ 3.7 mins	29	0.82 s	25	0.992	0.907
FA-4	46	~ 5.5 mins	49	0.75 s	41	0.995	0.910
FA-8	78	~ 9.3 mins	89	0.88 s	73	0.995	0.906
c432	276	~ 32.7 mins	691	0.91 s	259	0.997	0.652
PRE-ENC-1R	6651	~ 12.9 hrs	4920	3.62 s	1336	0.989	0.943
PRE-DEC-1R	6476	~ 12.7 hrs	4952	3.88 s	1160	0.990	0.898
PRE-ENC-2R	7986	~ 16.3 hrs	9839	4.47 s	2671	0.977	0.806
PRE-DEC-2R	7635	~ 15.0 hrs	9903	4.83 s	2319	0.984	0.809

2. **Pearson's correlation** between leakage scores of signals common to our tool and PLAN: Pearson's correlation gives us an indication of how closely related two sets of values are. The higher the Pearson's correlation score, the more closely in agreement the two sets of values are.

3. **Spearman's correlation** between leakage scores of signals common to our tool and PLAN: Spearman's correlation is very similar to Pearson's correlation, but it additionally takes into account the relative ordering of values in both the sets. The higher the Spearman's correlation score, the more closely in agreement the two sets of values are and the more closely their relative orderings match.

The results are summarised in Table 7.1. We can see that the time taken by our tool to complete the entire leakage calculation is orders of magnitude lesser than the time taken by PLAN for each of the designs. Moreover, the leakage scores given by our tool are comparable with those given by PLAN, as seen from the high value of Pearson's correlation scores. Also, the relative ordering of the signals with respect to the leakage scores is also well maintained, which is corroborated by the high values of the Spearman's correlation scores.

7.2 Evaluation of P2P-SoC

The P2P-SoC [Basak and Bhunia] is an open-source Verilog implementation of a System-on-Chip (SoC). The main modules in the SoC are as follows:

1. 128-bit Fast Fourier Transform (FFT) module
2. DLX RISC microprocessor, with a 5-stage pipeline and instruction cache
3. 128-bit AES encryption and decryption module
4. Serial Peripheral Interface (SPI) module for external communications
5. Power Management Controller (PMC) module
6. Security Policy Controller (SPC) module, consisting of the DLX microprocessor core, along with its own instruction cache and data cache
7. Debug Access Port (DAP) module, which controls the debug modules within each of the other modules of the SoC

An overview of the P2P-SoC is shown in Figure 7.1. On a high level, the input data is first processed by the FFT module, whose output drives the DLX processor, then the AES encrypts/decrypts the output of the processor, and sends it through the SPI module for external communication. The SPC module tracks the security control signals of the remaining modules throughout the execution.

Our tool completes the power side-channel vulnerability estimation of the P2P-SoC, having $\sim$ 3.6 million signals, in $\sim$ 6 hours. The module-wise analysis results are shown in Table 7.2. Thus, the tool scales well enough to very large designs. However, our tool does not support the inherent sequential constructs of such a design, since it poses a problem in the directed graph extraction step (in the first module) due to the presence of cyclic dependencies.

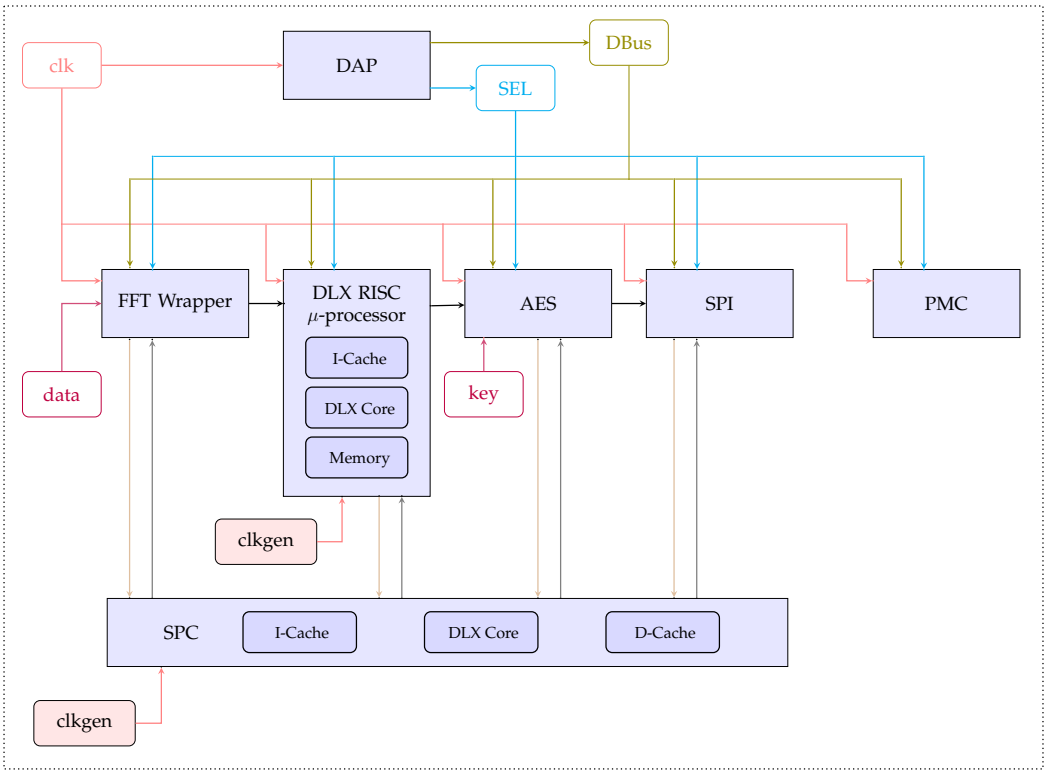


Figure 7.1: Overview of P2P-SoC

Table 7.2: P2P-SoC Module-wise Evaluation

Module	# Signals	Time Taken
FFT	~ 1.3 million	~ 2.5 hrs
DLX processor	~ 0.7 million	~ 5 min
AES	~ 0.3 million	~ 1 min
SPC	~ 1.3 million	~ 2.5 hrs
SPI	~ 20,000	~ 8 sec

CHAPTER 8

CONCLUSION AND FUTURE WORK

Our tool offers early and fine-grained power side-channel vulnerability estimation of digital designs, using a novel approach involving signal probabilities, and is able to scale up to evaluate very large designs such as a full-fledged SoC in a matter of a few hours. Some of the areas of future work could include building ML models of leakage in order to further speed up the analysis of larger designs, exploring more accurate power models, handling sequential constructs in the input RTL/netlist, and generating heatmaps of the leakages across different parts of a design for a better visualization.

APPENDIX A

Incremental Signal Probability Formulae

Lemma 1. *For a* `NOT` *gate with input A and output Out,*

$$SP(Out) = \Pr(Out = 1) = \Pr(A = 0) = 1 - \Pr(A = 1) = 1 - SP(A)$$

Lemma 2. *For an* `AND` *gate with inputs A, B (independent of each other) and output Out,*

$$SP(Out) = \Pr(Out = 1) = \Pr(A = 1 \cap B = 1) = \Pr(A = 1) \cdot \Pr(B = 1) = SP(A) \cdot SP(B)$$

Lemma 3. *For an* `OR` *gate with inputs A, B (independent of each other) and output Out,*

$$\begin{aligned} SP(Out) &= \Pr(Out = 1) = \Pr(A = 1 \cup B = 1) \\ &= \Pr(A = 1) + \Pr(B = 1) - \Pr(A = 1 \cap B = 1) \\ &= \Pr(A = 1) + \Pr(B = 1) - \Pr(A = 1) \cdot \Pr(B = 1) \\ &= SP(A) + SP(B) - SP(A) \cdot SP(B) \end{aligned}$$

Lemma 4. *For an* `XOR` *gate with inputs A, B (independent of each other) and output Out,*

$$\begin{aligned} SP(Out) &= \Pr(Out = 1) = \Pr((A = 1 \cap B = 0) \cup (A = 0 \cap B = 1)) \\ &= \Pr(A = 1 \cap B = 0) + \Pr(A = 0 \cap B = 1) \\ &= \Pr(A = 1) \cdot \Pr(B = 0) + \Pr(A = 0) \cdot \Pr(B = 1) \\ &= SP(A) \cdot (1 - SP(B)) + (1 - SP(A)) \cdot SP(B) \\ &= SP(A) + SP(B) - 2 \cdot SP(A) \cdot SP(B) \end{aligned}$$

AWARDS

1. Qualcomm Innovation Fellowship India 2020 for Winning Innovation Proposal "Instant Side-Channel Vulnerability Estimation of Digital Designs", under the guidance of Prof. Chester Rebeiro

REFERENCES

Basak, A. and **S. Bhunia** (). P2p-soc. URL `https://github.com/apdn/P2PSoC`.

Batina, L., **S. Bhasin**, **D. Jap**, and **S. Picek**, Csi nn: Reverse engineering of neural network architectures through electromagnetic side channel. *In 28th USENIX Security Symposium (USENIX Security 19)*. 2019.

Bhasin, S., **J.-L. Danger**, **S. Guilley**, and **Z. Najm**, Nicv: normalized inter-class variance for detection of side-channel leakage. *In 2014 International Symposium on Electromagnetic Compatibility, Tokyo*. IEEE, 2014.

Bogdanov, A., **L. R. Knudsen**, **G. Leander**, **C. Paar**, **A. Poschmann**, **M. J. Robshaw**, **Y. Seurin**, and **C. Vikkelsoe**, Present: An ultra-lightweight block cipher. *In International workshop on cryptographic hardware and embedded systems*. Springer, 2007.

Brier, E., **C. Clavier**, and **F. Olivier**, Correlation power analysis with a leakage model. *In International workshop on cryptographic hardware and embedded systems*. Springer, 2004.

Buhan, I., **L. Batina**, **Y. Yarom**, and **P. Schaumont** (2021). Sok: Design tools for side-channel-aware implementions. *arXiv preprint arXiv:2104.08593*.

Cooper, J., **E. DeMulder**, **G. Goodwill**, **J. Jaffe**, **G. Kenworthy**, **P. Rohatgi**, *et al.*, Test vector leakage assessment (tvla) methodology in practice. *In International Cryptographic Module Conference*, volume 20. 2013.

Demme, J., **R. Martin**, **A. Waksman**, and **S. Sethumadhavan**, Side-channel vulnerability factor: A metric for measuring information leakage. *In 2012 39th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2012.

Dubey, A., **R. Cammarota**, and **A. Aysu**, Maskednet: The first hardware inference engine aiming power side-channel protection. *In 2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2020.

Fahn, P. N. and **P. K. Pearson**, Ipa: A new class of power attacks. *In International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 1999.

Gierlichs, B., **L. Batina**, **P. Tuyls**, and **B. Preneel**, Mutual information analysis. *In International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2008.

Hansen, M. C., **H. Yalcin**, and **J. P. Hayes** (1999). Unveiling the iscas-85 benchmarks: A case study in reverse engineering. *IEEE Design & Test of Computers*, **16**(3), 72–80.

He, M. T., **J. Park**, **A. Nahiyan**, **A. Vassilev**, **Y. Jin**, and **M. Tehranipoor**, Rtl-psc: Automated power side-channel leakage assessment at register-transfer level. *In 2019 IEEE 37th VLSI Test Symposium (VTS)*. IEEE, 2019.

KF, M. A., **V. Ganesan**, **R. Bodduna**, and **C. Rebeiro**, Param: A microprocessor hardened for power side-channel attack resistance. *In 2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2020.

Kocher, P., **J. Jaffe**, and **B. Jun**, Differential power analysis. *In Annual international cryptology conference*. Springer, 1999.

Le, T.-H., **J. Clédière**, **C. Canovas**, **B. Robisson**, **C. Servière**, and **J.-L. Lacoume**, A proposition for correlation power analysis enhancement. *In International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2006.

Mangard, S., A simple power-analysis (spa) attack on implementations of the aes key expansion. *In International Conference on Information Security and Cryptology*. Springer, 2002.

Mangard, S., **E. Oswald**, and **T. Popp**, *Power analysis attacks: Revealing the secrets of smart cards*, volume 31. Springer Science & Business Media, 2008.

McCann, D., **E. Oswald**, and **C. Whitnall**, Towards practical tools for side channel aware software engineering:'grey box'modelling for instruction leakages. *In 26th {USENIX} Security Symposium ({USENIX} Security 17)*. 2017.

Moradi, A., **M. Kasper**, and **C. Paar**, Black-box side-channel attacks highlight the importance of countermeasures. *In Cryptographers' Track at the RSA Conference*. Springer, 2012.

Moradi, A., **D. Oswald**, **C. Paar**, and **P. Swierczynski**, Side-channel attacks on the bitstream encryption mechanism of altera stratix ii: facilitating black-box analysis using software reverse-engineering. *In Proceedings of the ACM/SIGDA international symposium on Field programmable gate arrays*. 2013.

Nahiyan, A., **J. Park**, **M. He**, **Y. Iskander**, **F. Farahmandi**, **D. Forte**, and **M. Tehranipoor** (2020). Script: A cad framework for power side-channel vulnerability assessment using information flow tracking and pattern generation. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, **25**(3), 1–27.

Oren, Y. and **A. Shamir** (2007). Remote password extraction from rfid tags. *IEEE Transactions on Computers*, **56**(9), 1292–1296.

Qin, Y. and **C. Yue**, Website fingerprinting by power estimation based side-channel attacks on android 7. *In 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*. IEEE, 2018.

Šijačić, D., **J. Balasch**, **B. Yang**, **S. Ghosh**, and **I. Verbauwhede** (2020). Towards efficient and automated side-channel evaluations at design time. *Journal of Cryptographic Engineering*, **10**(4), 305–319.

Slpsk, P., **P. K. Vairam**, **C. Rebeiro**, and **V. Kamakoti**, Karna: A gate-sizing based security aware eda flow for improved power side-channel attack protection. *In 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019.

Takamaeda-Yamazaki, S., Pyverilog: A python-based hardware design processing toolkit for verilog hdl. *In International Symposium on Applied Reconfigurable Computing*. Springer, 2015.

Tiri, K. and **I. Verbauwhede**, Securing encryption algorithms against dpa at the logic level: Next generation smart card technology. *In International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2003.

Veshchikov, N., Silk: high level of abstraction leakage simulator for side channel analysis. *In Proceedings of the 4th Program Protection and Reverse Engineering Workshop*. 2014.

Veshchikov, N. and **S. Guilley**, Use of simulators for side-channel analysis. *In 2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2017.

Zoni, D., **A. Barenghi**, **G. Pelosi**, and **W. Fornaciari** (2018). A comprehensive side-channel information leakage analysis of an in-order risc cpu microarchitecture. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, **23**(5), 1–30.